Documentación y diseño técnico

SOCIAL-SIM Chat: Entrenador conversacional y learning analytics para menores con TEA/Asperger

Módulo C088 · Proyecto de Inteligencia Artificial y Big Data

Curso 2026–2027 Núcleo Formativo 02 (NF2) · Semestre 2609 ·

Fecha límite: 14/10/2026 Equipo: Álvaro García Puche y Marcos Expósito Flox

Índice

1. Título del proyecto e integrantes 3
2. Resumen ejecutivo 3
3. Descripción funcional del sistema 3
4. Alcance definitivo (IN / OUT) 4
5. Arquitectura del sistema (alto nivel) 5
6. Tecnologías seleccionadas 6
7. Modelo de datos 7
8. Datos 9
9. Ética, legalidad y accesibilidad 11
10. Backlog inicial 12
11. Plan de trabajo 13
12. Estrategia de desarrollo 13
13. Riesgos técnicos actualizados 14
14. Repositorio del proyecto 15
Anexo A. Evaluación del clasificador y regla de alerta 16
Anexo B. Declaración de uso de inteligencia artificial y autoría de recursos 17

1. Título del proyecto e integrantes

Proyecto: SOCIAL-SIM Chat: Entrenador conversacional y learning analytics para menores con TEA/Asperger

Integrante	Rol principal
Marcos Expósito Flox	Plataforma + ingeniería ML (repositorio, entorno reproducible, backend FastAPI, base de datos, generador de datos sintéticos, pipeline de features y entrenamiento del clasificador)
Álvaro García Puche	Dominio pedagógico + Datos + BI (guiones y personajes, especificación de perfiles simulados y trayectorias, definición de métricas y etiquetas, dashboard del terapeuta, coordinación de memoria y defensa)

2. Resumen ejecutivo

Los terapeutas que trabajan con menores con TEA (6-12 años) no cuentan con ninguna herramienta en español que combine entrenamiento conversacional de habilidades pragmáticas (metáforas, sarcasmo, deixis, preguntas) con analítica de progreso entre sesiones. Su ajuste de tratamiento depende hoy de la percepción subjetiva de 1-2 sesiones semanales, sin serie temporal objetiva de latencias, aciertos o evolución.

SOCIAL-SIM Chat es un sistema local de entrenamiento conversacional para estas competencias, que registra cada interacción del menor con un chatbot guiado (LLM local vía Ollama) y entrega al terapeuta un panel con métricas por sesión y alertas tempranas de estancamiento, generadas por un clasificador propio (Progreso / Estancamiento / Regresión) entrenado por el equipo sobre un corpus sintético con evolución conocida por diseño.

El valor que aporta es doble: para el menor, práctica adicional entre sesiones con personajes estables y predecibles; para el terapeuta, una vista objetiva y longitudinal de la evolución que hoy no existe, procesada íntegramente en local para no exponer datos de salud de un menor a servicios en la nube.

3. Descripción funcional del sistema

Qué hace exactamente la aplicación

- El menor conversa con un chatbot (uno de 2-3 personajes estables) que le presenta guiones de diálogo centrados en una competencia pragmática (metáfora, sarcasmo, deixis o preguntas recíprocas).
- Cada turno de conversación se registra como un evento estructurado: latencia de respuesta, tropo/figura presentada, puntuación de la respuesta (0 / 1 / 2, asignada por el backend; ver sección 7), ayudas solicitadas e iniciativa del menor.
- El backend calcula cinco métricas agregadas por sesión (aMLU, latencia pragmática, IRC%, PEPE, %DET; fórmulas en la sección 7). Un clasificador entrenado por el equipo sobre un corpus sintético con trayectorias conocidas toma la evolución de esas métricas en las últimas 4 sesiones y predice el estado del alumno: Progreso, Estancamiento o Regresión.
- El terapeuta accede a un dashboard (Streamlit) con selector de alumno, métricas en tiempo real, alertas de estancamiento, gráficos de evolución longitudinal frente a un baseline neurotípico simulado, y una consola de transcripciones editable donde puede corregir manualmente la puntuación de un turno.
- El terapeuta puede añadir notas clínicas y consolidar la configuración de la siguiente sesión.

Flujo general de uso

1. El terapeuta selecciona al alumno (simulado) en el panel.
2. El menor realiza una sesión de práctica con el chatbot sobre una competencia concreta.
3. El sistema captura los eventos de la sesión y calcula las métricas agregadas.
4. El clasificador ML predice el estado de la sesión y lo muestra en el dashboard; la alerta de estancamiento salta tras 3 sesiones consecutivas predichas como no-progreso (sección 8).
5. El terapeuta revisa las métricas, la alerta, y opcionalmente corrige o anota, y decide el guion de la siguiente sesión (con apoyo de la recomendación del sistema).

Casos principales de usuario

- Un menor practica una competencia pragmática concreta con el chatbot.
- Un terapeuta revisa la evolución de un alumno a lo largo de varias sesiones.

- Un terapeuta recibe una alerta de estancamiento en una competencia y ajusta el plan de intervención.

Qué puede y qué no puede hacer el sistema

- **Puede:** practicar 4 competencias pragmáticas con guiones cerrados; registrar y almacenar eventos por turno; calcular métricas agregadas por sesión; predecir Progreso/Estancamiento/Regresión con un modelo entrenado sobre datos sintéticos; mostrar un dashboard con alertas y evolución longitudinal; ejecutarse íntegramente en local (incluye un modo de LLM simulado sin GPU).
- **No puede:** generar contenido conversacional libre fuera de los guiones definidos; entrenar o validar clínicamente el modelo con datos reales de menores (fuera de alcance de este curso); sustituir el criterio clínico del terapeuta — la predicción es un apoyo a la decisión, no un diagnóstico; escalar a un pipeline Kafka+Spark en producción (queda documentado, no implementado).

4. Alcance definitivo (IN / OUT)

Incluye (funcionalidades mínimas obligatorias):

- Entrenador conversacional con 2-3 personajes y guiones de 4 competencias pragmáticas (metáfora, sarcasmo, deixis, preguntas) sobre un LLM local (Ollama, 8B-14B).
- Captura de logs estructurados por turno.
- Base de datos PostgreSQL (`student_profile_sim`, `session_data` y `turn_data`).
- Clasificador propio (entrenado por el equipo) que detecta Progreso / Estancamiento / Regresión sobre corpus sintético.
- Dashboard con métricas básicas y alerta (Streamlit).
- Entorno reproducible con Docker Compose (incluye modo de LLM simulado, sin GPU).

Funcionalidades opcionales:

- Consola de transcripciones editable con corrección manual del terapeuta.
- Formulario de notas clínicas y recomendación adaptativa para la siguiente sesión.

No incluye (elementos descartados explícitamente):

- Pipeline Kafka+Spark en producción (se documenta el diseño de cómo escalaría, no se implementa).
- Modelo LSTM de series temporales (queda como evolución futura frente al RandomForest/Regresión Logística inicial).
- Despliegue en Kubernetes.
- Integración con datos reales de menores (fuera de alcance por motivos éticos/legales en el marco de este curso).
- Avatar 3D, voz y grabación de sesiones, y sesiones guiadas en persona por un especialista (en el sistema, el terapeuta del dashboard ocupa ese papel de supervisión).
- Grupos neurotípicos reales: el baseline neurotípico del dashboard es simulado (sección 8).

5. Arquitectura del sistema (alto nivel)

Diagrama de arquitectura:

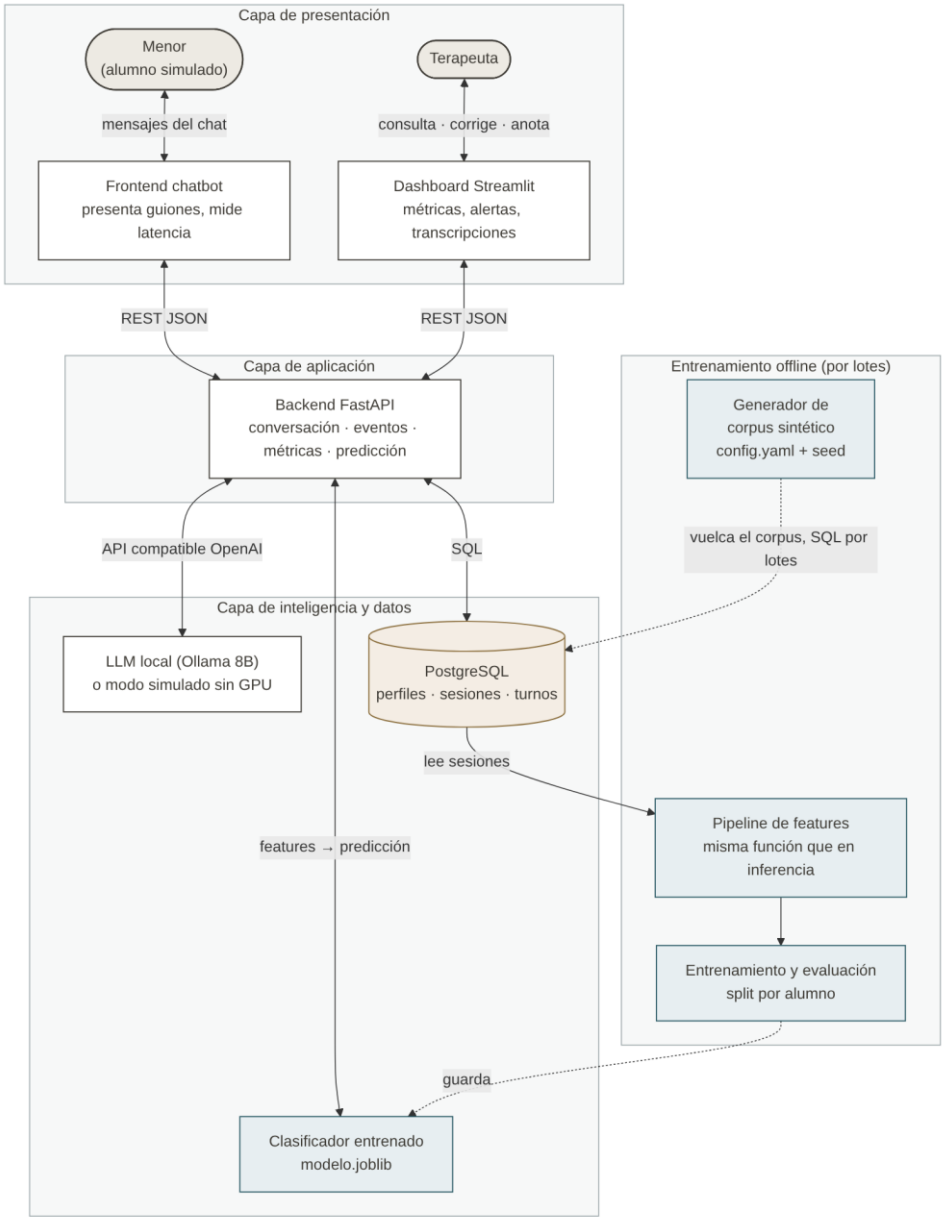


Diagrama de arquitectura de SOCIAL-SIM Chat

(Generado con Mermaid (versión 2: sesión y entrenamiento separados, flechas rotuladas); fuente editable en `arquitectura.mmd` junto a este documento. En el repositorio debe colocarse en `docs/arquitectura.png`, según la estructura exigida en la sección 14.)

Descripción de cada componente:

- **Frontend (chatbot menor):** interfaz de conversación con el LLM local, presenta los guiones y captura la interacción del menor turno a turno.
- **Backend:** FastAPI, expone la conversación, registra eventos y sirve las predicciones del modelo ML al dashboard. Accede al LLM mediante una interfaz compatible con OpenAI, de forma que el motor local se pueda sustituir sin cambiar el resto del sistema.
- **Base de datos:** PostgreSQL, tablas `student_profile_sim` (perfil y trayectoria del alumno simulado), `session_data` (métricas por sesión) y `turn_data` (eventos por turno).

- **Servicios ML:** generador de corpus sintético con trayectorias conocidas por diseño, que emite dos ficheros (`corpus_train`, lo que ve el modelo, y `corpus_meta`, competencia/arquetipo/inicio del cambio/ambiguo, que no entra al modelo) → features de ventana sobre las 5 métricas (últimas 4 sesiones: media, pendiente normalizada y diferencia respecto a las 3 primeras sesiones; 15 features) → entrenamiento y evaluación de un RandomForest (validación cruzada agrupada por alumno simulado, no por sesión) → carga del modelo desde FastAPI para predicción en cada sesión, con la misma función de features que en entrenamiento.
- **Pipeline de datos:** PostgreSQL como almacén único (decisión de arquitectura Big Data confirmada como válida por el docente); se documenta cómo escalaría a Kafka (ingesta) + Spark (procesamiento) en producción, sin implementarlo en este prototipo, priorizando un sistema funcional y reproducible con un volumen de unos 9.400 registros (1.500 sesiones y 7.875 turnos).
- **Dashboard:** Streamlit, con barra lateral de selección de alumno/sesión, métricas en tiempo real, bloque de alertas, gráficos longitudinales (Plotly/Altair) y consola de transcripciones editable (`st.data_editor`).
- **Otros servicios:** modo de LLM simulado para ejecutar y evaluar el prototipo sin GPU; Docker Compose para levantar todo el stack con un solo comando.

6. Tecnologías seleccionadas

Tecnología	Uso	Por qué se ha elegido	Alternativa descartada
Python + FastAPI	Backend	Estándar para servir APIs de IA/ML, integración directa con el resto del stack Python	Node.js/Express (menos natural para servir el modelo ML)
Ollama (LLM local, 8B-14B)	Motor conversacional	Coherente con la soberanía del dato: menores + datos de salud no deben salir a la nube	API de LLM en la nube (OpenAI, Anthropic) — descartada por privacidad
PostgreSQL	Base de datos	Simplicidad y madurez frente a Kafka+Spark para un equipo de 2 personas y un volumen de datos moderado	Kafka + Spark en producción (documentado como evolución, no implementado por bajo valor real con este volumen)
scikit-learn	Clasificador ML (RandomForest / Regresión Logística)	Modelos simples, interpretables y suficientes para un corpus sintético de tamaño moderado	LSTM (descartado para esta fase, queda como evolución futura hacia series temporales)
Streamlit	Dashboard del terapeuta	Prototipado rápido de paneles con componentes de datos interactivos, sin necesitar un frontend a medida	React/Next.js a medida (mayor coste de desarrollo para un MVP académico)
Docker Compose	Entorno reproducible	El prototipo se levanta con un solo comando en cualquier equipo, incluye modo LLM simulado sin GPU	Kubernetes (descartado por sobredimensionado para el alcance del curso)

7. Modelo de datos

Diagrama entidad-relación:

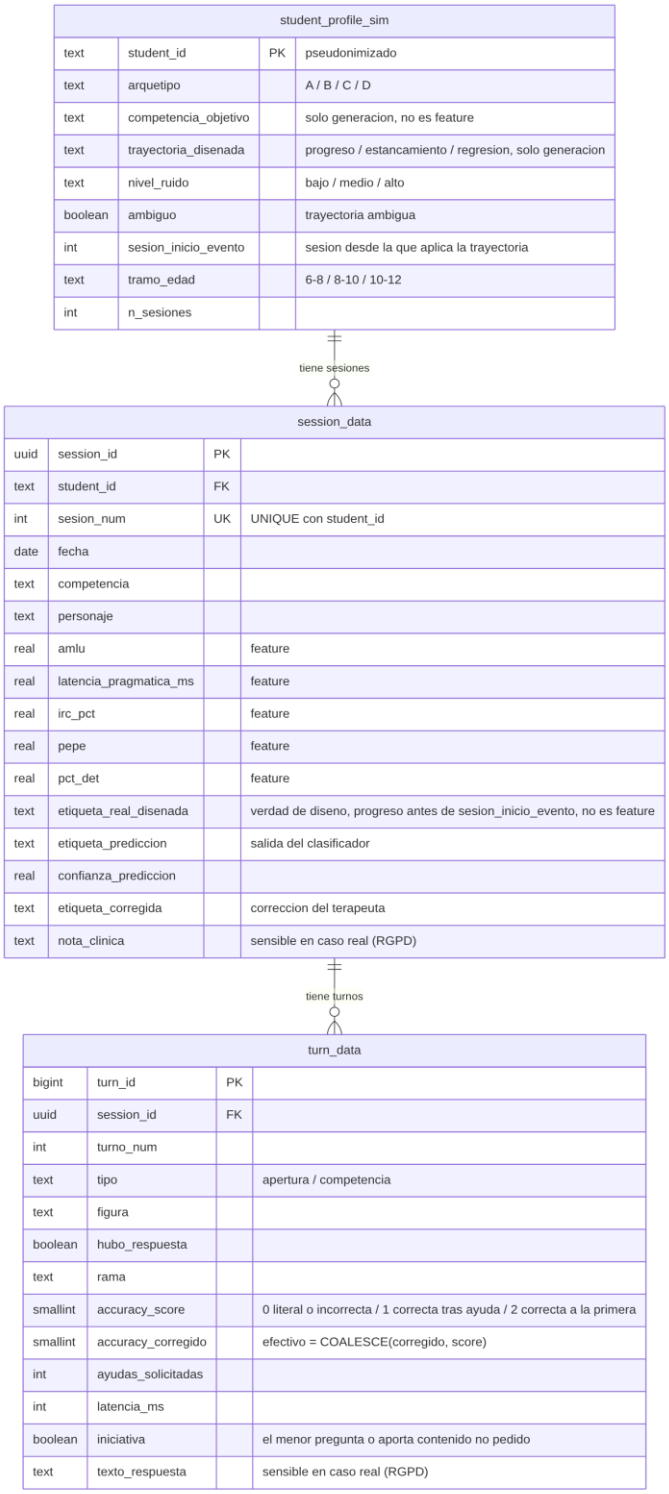


Diagrama entidad-relación de SOCIAL-SIM Chat

(Generado con Mermaid (versión 2); fuente editable en `er.mmd`. Relaciones 1:N: un `student_profile_sim` tiene muchas `session_data` (UNIQUE `(student_id, sesion_num)`) y una `session_data` tiene muchos `turn_data`. Los turnos se normalizan en tabla propia para poder calcular las métricas de sesión desde los eventos y para que el terapeuta corrija el `accuracy_corregido` turno a turno.)

Explicación de tablas:

- **`student_profile_sim`** — perfil del alumno simulado y su trayectoria de evolución conocida por diseño (p. ej. "se estanca en sarcasmo desde la sesión 8"). Sirve para generar sesiones con variabilidad realista alrededor de esa trayectoria.
  - Campos principales: `student_id` (PK, pseudonimizado), `arquetipo`, `competencia_objetivo` y `trayectoria_disenada` (progreso/estancamiento/regresión; **solo para generar, no son features**), `sesion_inicio_evento` (sesión desde la que aplica la trayectoria), `nivel_ruido`, `ambiguo`, `tramo_edad`, `n_sesiones`.
- **`session_data`** — métricas agregadas por sesión.
  - Campos principales: `session_id` (PK), `student_id` (FK), `sesion_num`, `fecha`, `competencia`, `personaje`, `amlu`, `latencia_pragmatica_ms`, `irc_pct`, `pepe`, `pct_det`, `etiqueta_real_disenada` (verdad de diseño para evaluar el modelo, no es feature), `etiqueta_prediccion`, `confianza_prediccion` (probabilidad del modelo, no calibrada), `etiqueta_corregida` (terapeuta), `nota_clinica`.
- **`turn_data`** — evento por turno del menor.
  - Campos principales: `turn_id` (PK), `session_id` (FK), `turno_num`, `tipo` (`apertura` / `competencia`), `figura`, `hubo_respuesta`, `rama`, `accuracy_score`, `accuracy_corregido`, `ayudas_solicitadas`, `latencia_ms`, `iniciativa` (boolean: el menor pregunta o aporta contenido que el guion no pedía; necesario para el IRC %), `texto_respuesta`. `texto_respuesta` y `nota_clinica` serían datos sensibles en un caso real (sección 9).

Puntuación de la respuesta (`accuracy_score`). 0 = literal o incorrecta incluso tras la ayuda; 1 = correcta tras la ayuda del guion; 2 = correcta a la primera. **La asigna el backend automáticamente.** El método (reglas del guion, valoración con el modelo local o una combinación de ambos) se fija en NF3 y se evalúa contra un conjunto de respuestas etiquetadas, midiendo cuántas respuestas buenas reciben 2 y cuántas malas reciben 2; cada turno guarda con qué versión del evaluador se puntuó. Sea cual sea el método, el modo simulado sin GPU debe poder puntuar. **El terapeuta puede corregirla** en el dashboard (`accuracy_corregido`); el valor original se conserva y las métricas usan el efectivo, `COALESCE(accuracy_corregido, accuracy_score)`. En el corpus sintético no la asigna nadie: la derivaría el generador a partir de PEPE y %DET de la sesión (script aparte, `generar_turnos.py`).

Diccionario de variables (features del clasificador):

Variable	Descripción
aMLU	Longitud Media de Enunciado Adaptativo — progreso del desarrollo sintáctico
Latencia pragmática (LPP)	Mediana de latencia de respuesta en milisegundos
IRC %	Índice de Reciprocidad Conversacional — % de iniciativa conversacional activa del menor
PEPE	Eficacia Pragmática Ponderada — aciertos pragmáticos acumulados en el nivel de dificultad actual
%DET	Determinismo sintáctico (RQA) — convergencia de la conducta conversacional sesión a sesión

Fórmulas operativas por sesión (propuesta operativa para el prototipo; la definición teórica de cada métrica es la del TFM y, si difiere, manda el TFM). T = turnos del menor con respuesta; E = turnos de tipo `competencia` (los evaluados).

Métrica	Fórmula	Rango
aMLU	media de palabras de `texto_respuesta` sobre T, excluyendo la ecolalia inmediata	1 a 7
LPP	**mediana** de `latencia_ms` sobre E (resiste atípicos)	400 a 9.000 ms
IRC %	100 × (turnos de T con `iniciativa`) / |T|	0 a 100
PEPE	$\Sigma\, w_t \cdot \text{score}(t) / \Sigma\, w_t$ sobre E; w_t = peso de dificultad del turno (1 si el guion no define niveles)	0 a 2

Métrica	Fórmula	Rango
%DET	% de puntos recurrentes que forman líneas diagonales de longitud ≥ 2 en la matriz de recurrencia de los `score` de E de las últimas 3 sesiones de la misma competencia (RQA): DET = 100 · $\Sigma_{l\geq 2}\, l\cdot P(l) / \Sigma_{l\geq 1}\, l\cdot P(l)$	0 a 100

Con secuencias tan cortas (9-12 símbolos) el %DET es ruidoso; se declara como limitación. En el corpus sintético las métricas de sesión se generan directamente; en el sistema real se calculan desde los turnos con estas fórmulas, que se validan con tests unitarios (sección 12). La propuesta del proyecto define el %DET como la regularidad del patrón conversacional del menor frente a una banda neurotípica, y el dashboard lo grafica sesión a sesión, pero ningún documento del proyecto fija la secuencia ni la fórmula; la secuencia elegida es, por tanto, una decisión de diseño del equipo, coherente con esa definición.

Relaciones: un alumno simulado (`student_profile_sim`) tiene muchas sesiones (`session_data`) y cada sesión muchos turnos (`turn_data`). La etiqueta real de cada sesión se deriva de la trayectoria diseñada del alumno, no de una única métrica, por lo que el clasificador debe combinar las cinco señales.

Regla de etiqueta por sesión: `progreso` si la sesión es anterior a `sesion_inicio_evento` o la trayectoria del alumno es progreso; desde ese inicio, la etiqueta de la trayectoria (estancamiento o regresión). La etiqueta se fija por diseño y no se recalcula desde las métricas ruidosas (así coincide con los perfiles B y C de `Perfiles_Alumno_Simulado.md`: progreso hasta el inicio del cambio y estancamiento o regresión después).

Claves: `student_id` como clave foránea en `session_data`, `session_id` como clave primaria de sesión.

8. Datos

- **Fuente:** corpus sintético generado por el propio equipo (perfiles de alumno simulado con evolución conocida de antemano). No existe ningún dataset público de transcripciones de menores con TEA, ni sería legalmente accesible en el plazo del curso.
- **Licencia:** datos de generación propia; sin restricciones de licencia externa (no se reutiliza ningún corpus de terceros).
- **Tipo de datos:** texto conversacional (guiones + turnos) + metadatos de interacción (latencia, tropo, resultado) + métricas agregadas por sesión.
- **Volumen (cerrado, validado con el generador real de Marcos, PR #2):** los 4 perfiles de `Perfiles_Alumno_Simulado.md` son arquetipos de diseño; el generador (`src/generador/`, parametrizado por `config.yaml`) produce varias instancias por competencia (el arquetipo solo fija la competencia, no el desenlace) con variabilidad aleatoria distinta. Configuración por defecto: **25 alumnos por competencia (100 en total) × 15 sesiones = 1.500 sesiones**, de las que 1.200 (sesiones 4-15) entran al clasificador, porque las 3 primeras no tienen ventana completa. Con la estructura de guiones (2 turnos de apertura + 4 evaluables en Metáfora y 3 en las otras tres competencias) son **7.875 eventos de turno**, que se derivan de las métricas de sesión con un script aparte (`generar_turnos.py`, posterior a NF2) y alimentan la base de datos y el dashboard, no el clasificador. Reproducible con la misma semilla. Los CSV del generador usan `alumno_id`, `sesion`, `etiqueta` e `inicio_cambio`; al cargarlos en PostgreSQL se mapean a `student_id`, `sesion_num`, `etiqueta_real_disenada` y `sesion_inicio_evento` del ER.
- **Cómo se genera el corpus (diseño):**
  - *Tendencia, no regla:* la trayectoria de cada alumno (progreso / estancamiento / regresión) se muestrea según su competencia, para que la competencia no determine el desenlace: Sarcasmo 0,70 estancamiento / 0,20 progreso / 0,10 regresión; Preguntas recíprocas 0,20 / 0,10 / 0,70; Metáfora 0,25 / 0,60 / 0,15; Deixis 0,30 / 0,55 / 0,15 (estancamiento / progreso / regresión). Son **supuestos de diseño basados en razonamiento clínico, no tasas empíricas** (declarado como limitación).
  - *Inicio del cambio variable* por alumno (sesión 4 a 12); antes de él todos progresan.
  - *Realismo:* ruido autorregresivo más ruido de medición, recaídas puntuales en las cinco métricas, un 8 % de alumnos con trayectoria ambigua (incluye cambio de signo) y un 2 % de ruido en las etiquetas.

  - *Sin atajos:* la competencia, el arquetipo y el nº de sesión no son features; el generador emite `corpus_train` (lo que ve el modelo) y `corpus_meta` (lo que no).
  - Distribución de clases resultante: 66 % progreso / 19 % estancamiento / 15 % regresión sobre las 1.500 sesiones (59 % progreso en las sesiones 4-15), por lo que se usa `class_weight="balanced"` y F1-macro como métrica principal.
- **Arranque en frío y ventana (reglas de diseño):** las sesiones 1-3 son la evaluación inicial: sin predicción, el dashboard muestra «datos insuficientes». El clasificador opera desde la 4.ª sesión y la primera alerta posible llega en la 6.ª. La ventana de 4 sesiones se calcula por pareja (alumno, competencia). El %DET exige al menos 3 sesiones del alumno en la competencia; sobre sesiones reales con menos, el proxy «% de turnos con 2» no equivale a él y no se usa como entrada del clasificador. Limitación del generador: el cambio nunca empieza antes de la sesión 4, así que la línea base sale limpia por construcción; en datos reales podría estar ya contaminada.
- **Baseline neurotípico (historia 8):** el generador solo produce los 4 arquetipos con TEA, no hay datos neurotípicos. La banda del dashboard se define con **valores de referencia por tramo de edad (6-8, 8-10, 10-12)** fijados por diseño y rotulados en pantalla como «referencia simulada, no medida». Mejora opcional para NF3: un grupo `control` en el generador (nivel alto y estable, ruido bajo), excluido del entrenamiento.
- **Frecuencia de actualización (propuesta cerrada por Álvaro):** generación **por lotes**, no en streaming (coherente con no usar Kafka). El corpus se regenera cuando cambie el generador y es estático una vez fijado para entrenar y evaluar. Con datos reales, la frecuencia pasaría a ser por sesión registrada.
- **Calidad esperada:** controlada por diseño (la trayectoria de cada alumno simulado es conocida de antemano), con variabilidad realista añadida para que el clasificador no dependa de una regla trivial sobre una sola métrica. Se verifica con comprobaciones antifuga (modelo tramposo y etiquetas permutadas, Anexo A). El corpus original daba 100 % de acierto (trivial); el rediseñado da un problema difícil pero resoluble.
- **Riesgos de datos:** al ser sintéticos, no capturan la variabilidad real de un menor con TEA; existe riesgo de que el corpus no represente bien casos límite reales. Mitigación: documentar explícitamente esta limitación y reservar la validación clínica con datos reales para una fase posterior fuera del alcance de este curso.
- **¿Son reales, simulados o mockeados?** Son **simulados**: cada alumno tiene una trayectoria de evolución diseñada de antemano (progreso/estancamiento/regresión desde una sesión concreta, variable por alumno), y las cinco métricas se derivan de esa trayectoria con variabilidad realista, no valores mockeados sin lógica subyacente.
- **Criterio cuantitativo de éxito del clasificador (cerrado el 11/10/2026 con la medición de Marcos; responde al feedback del docente sobre Fase 0, 27/09/2026).** RandomForest (300 árboles, `class_weight="balanced"`), validación cruzada agrupada por alumno (`StratifiedGroupKFold`, k=5) sobre 10 semillas del generador. Los umbrales se miran sobre la **media de la validación cruzada**, no sobre una partición suelta.

Las cifras corresponden a las versiones fijadas en el requirements.txt del repositorio (scikit-learn 1.9.1, numpy 2.5.3, pandas 3.0.5). Con scikit-learn 1.6.1 el corpus es idéntico, pero el RandomForest reparte distinto: el F1-macro no cambia (0,78) y el recall de estancamiento baja a 0,650 (0,6497 en la medición de Marcos), unas milésimas por debajo del umbral y dentro de la dispersión entre semillas (± 0,05).

Criterio	Umbral	Medido	Margen
F1-macro (3 clases)	≥ 0,75	0,78 ± 0,02	+0,03
Precisión de estancamiento	≥ 0,65	0,69 ± 0,05	+0,04
Recall de estancamiento	≥ 0,65	0,69 ± 0,05	+0,04
Supera a la heurística (F1-macro)	≥ 0,05 de ventaja	0,78 frente a 0,68	+0,10
Matriz de confusión completa	en la memoria	ver abajo	—

El ± de la tabla es la variación **entre semillas**; entre particiones sueltas es mayor (± 0,05 en F1 y ± 0,10 en recall), y ese es el argumento para evaluar sobre la media. Por semilla, el F1-macro ≥ 0,75 se cumple en 7 de 10 (las tres que fallan quedan en 0,744-0,750) y la precisión y el recall de estancamiento ≥ 0,65 en 8 de 10; la semilla por defecto (42) cumple todo (Anexo A).

El recall de estancamiento se revisó de 0,80 a 0,65 **después de la primera medición**, por el efecto de la transición (en las 2-3 primeras sesiones tras el cambio el acierto es 0,52 frente a 0,85 después); no es un ajuste arbitrario. Umbral y nivel de ruido del generador están acoplados: el criterio solo tiene sentido junto al ruido declarado. La matriz de confusión completa, las comprobaciones antifuga y el análisis de sensibilidad están en el **Anexo A**.

- **Regla de alerta (implementada):** el dashboard alerta cuando el clasificador predice **no-progreso (estancamiento o regresión) en 3 sesiones consecutivas**. Medida a nivel de alumno: el 97 % de los alumnos con cambio llega a alertar, con retraso mediano de 3 sesiones y un 9 % de alumnos que progresan con alguna falsa alerta (Anexo A).

- **Disclaimer de validez (obligatorio en la memoria, feedback del docente):** los resultados de rendimiento del clasificador se obtienen íntegramente sobre un corpus sintético generado por el propio equipo, con etiquetas conocidas por diseño. Miden si el modelo aprende los patrones programados por el equipo y validan la arquitectura (extracción de features, entrenamiento, separación por alumno, integración con el dashboard) de forma reproducible, pero **no constituyen evidencia de eficacia clínica ni de validez sobre pacientes reales** — eso requeriría datos reales, supervisión clínica y un proceso de validación fuera del alcance de este prototipo académico (ver "Visión y recorrido", fase V2). Los casos ambiguos, el ruido y las probabilidades de emparejamiento competencia-trayectoria están puestos a propósito y son supuestos de diseño, no tasas empíricas.

9. Ética, legalidad y accesibilidad

- **Presencia/ausencia de datos personales (PII):** no se usan datos reales de menores. El corpus es 100% sintético; los identificadores de alumno simulado son pseudonimizados y no corresponden a personas reales.
- **Cumplimiento GDPR:** al no tratarse datos personales reales (corpus 100 % sintético, sin menores implicados), el sistema **no procesa datos de carácter personal** en el sentido del RGPD durante NF2-NF4. Para una eventual validación clínica (fuera del alcance de este curso) serían obligatorios: consentimiento informado de las familias y del menor, aprobación de un comité ético o centro, minimización y pseudonimización desde el diseño (`texto_respuesta` y `nota_clinica` serían datos sensibles), cifrado en reposo y en tránsito, y almacenamiento exclusivamente local/on-premise, coherente con el LLM local (Ollama).
- **Riesgos de sesgo:** el corpus sintético refleja los supuestos del equipo sobre cómo evoluciona un alumno con TEA (por ejemplo, que Sarcasmo tiende al estancamiento y Preguntas recíprocas a la regresión), lo que puede introducir sesgos si esos supuestos no son representativos de la variabilidad real. Se mitiga tratando el emparejamiento como tendencia y no como regla, y se documenta como limitación explícita del prototipo académico. Otras limitaciones declaradas: no se ha medido el acuerdo entre la puntuación automática y el terapeuta; las reglas de puntuación solo valen para guiones cerrados y, al basarse en palabras clave, pueden penalizar paráfrasis válidas (empujaría a falsos no-progreso; se medirá en NF3); el %DET es ruidoso con secuencias cortas.
- **Apego, dependencia y rechazo del asistente:** un chatbot estable puede generar apego, dependencia comunicativa unidireccional o rechazo. Medidas: el terapeuta decide el uso; límite de duración y de sesiones por semana; personaje sin lenguaje afectivo; es práctica entre sesiones de terapia, no un sustituto; si el menor lo rechaza, se suspende y se deriva al terapeuta.
- **Qué se evalúa:** el clasificador resume la trayectoria de las métricas de uso para apoyar la decisión del terapeuta; no evalúa capacidades del menor ni diagnostica. La eficacia de la herramienta requeriría una validación futura con datos reales.

- **Medidas de mitigación:** separación train/test por alumno simulado (no por sesión) para evitar sobreajuste; heurística simple de referencia (regla sobre la pendiente de PEPE en las últimas sesiones) para contrastar el comportamiento del modelo; comprobaciones antifuga (modelo tramposo y etiquetas permutadas); reserva explícita de la validación clínica para una fase posterior con datos reales y supervisión de un centro.
- **Accesibilidad básica prevista (frontend/dashboard):** las alertas (`st.success`/`st.warning`/`st.error`) llevan siempre icono y etiqueta textual («Progreso óptimo» / «Alerta de estancamiento»), no solo color; fuente mínima de 16 px en el panel del terapeuta con componentes nativos de Streamlit (contraste AA razonable por defecto, sin CSS de bajo contraste); navegación por teclado con los componentes nativos (`st.selectbox`, `st.slider`, `st.button`, `st.data_editor`) sin JavaScript custom. No se persigue conformidad WCAG completa (fuera de alcance de NF2-NF4): es una buena práctica mínima verificable, no una certificación.

10. Backlog inicial

Priorización (MoSCoW), propuesta por Álvaro y validada por Marcos el 13/10 con dos cambios (la alerta pasa a Must y se añade la predicción en el backend):

- **Must (imprescindibles para el mínimo de NF3):** historias 1, 2, 3, 4, 5, 7, 11, 12 — el ciclo mínimo end-to-end (conversar → registrar evento → generar corpus → calcular métricas → entrenar → predecir y alertar → levantar el stack). La alerta (7) es para lo que existe el clasificador; en su versión mínima cuesta poco.
- **Should (valor alto, no bloquean la demo mínima):** historia 6 — dashboard con las métricas de la sesión.
- **Could (mejoran la demo pero son recortables si el calendario aprieta):** historias 8, 9, 10 — baseline neurotípico, consola editable, notas clínicas.
- **Won't (este ciclo):** el grupo de control neurotípico real (el baseline del dashboard es simulado, sección 8). El escalado a Kafka+Spark ya está documentado en la sección 5.

#	Historia de usuario	Criterios de aceptación
1	Como menor, quiero conversar con un personaje del chatbot sobre una competencia pragmática, para practicar entre sesiones.	El sistema presenta un guion de la competencia seleccionada; el menor puede responder y recibir una réplica generada por el LLM local; se cierra el turno de forma completa.
2	Como sistema, quiero registrar cada turno de conversación como un evento estructurado, para poder calcular métricas después.	Cada turno genera un registro con `figura`, `accuracy_score`, `ayudas_solicitadas`, `iniciativa` y `latencia_ms`, persistido en `turn_data`.
3	Como equipo, queremos generar perfiles de alumno simulado con una trayectoria de evolución conocida, para entrenar y evaluar el clasificador.	El generador produce alumnos con trayectorias distintas (progreso/estancamiento/regresión) y variabilidad realista, es reproducible con la misma semilla y supera las comprobaciones antifuga del Anexo A.
4	Como sistema, quiero calcular las cinco métricas agregadas por sesión (aMLU, LPP, IRC%, PEPE, %DET), para alimentar el clasificador.	Las métricas se calculan al cerrar una sesión con las fórmulas de la sección 7, validadas con los tests de la sección 12, y quedan almacenadas en `session_data`.
5	Como equipo, queremos entrenar un clasificador que prediga Progreso/Estancamiento/Regresión, para generar la alerta del terapeuta.	La media de la validación cruzada por alumno cumple los umbrales de la sección 8 y supera a la heurística; se documentan las métricas obtenidas.
6	Como terapeuta, quiero ver un dashboard con las métricas de la sesión actual de un alumno, para valorar su evolución.	El dashboard muestra las cinco métricas con indicador de variación respecto a la sesión anterior.
7	Como terapeuta, quiero recibir una alerta visual cuando el modelo detecta estancamiento, para poder ajustar el plan de intervención.	Se muestra un bloque de alerta (éxito/advertencia/error) con la probabilidad del modelo y la competencia afectada, tras 3 sesiones consecutivas predichas como estancamiento o regresión (regla de la sección 8).
8	Como terapeuta, quiero ver la evolución longitudinal de un alumno frente a un baseline neurotípico simulado, para contextualizar el progreso.	Existe al menos un gráfico temporal (%DET u otra métrica) comparando al alumno con la banda de referencia neurotípica simulada (valores de diseño por tramo de edad).
9	Como terapeuta, quiero revisar la transcripción completa de una sesión y poder corregir manualmente la puntuación de	La consola de transcripciones permite editar el `accuracy_corregido` de un turno (el `accuracy_score` original se conserva) y persistir el cambio.

#	Historia de usuario	Criterios de aceptación
	un turno, para corregir errores del script de puntuación que producen falsos positivos.	
10	Como terapeuta, quiero añadir notas clínicas a una sesión, para dejar constancia de observaciones subjetivas.	Existe un campo de texto libre asociado a la sesión que se guarda junto al resto de datos.
11	Como equipo, queremos levantar todo el sistema con un solo comando y sin necesidad de GPU, para poder ejecutar y evaluar el prototipo en cualquier equipo.	`docker compose up` levanta backend, base de datos y dashboard; existe un modo de LLM simulado funcional sin GPU.
12	Como sistema, quiero predecir el estado de cada sesión al cerrarla con el modelo entrenado, usando la misma función de features que en el entrenamiento, para alimentar el dashboard y la alerta.	La predicción y su probabilidad se guardan en `session_data`; las sesiones 1-3 muestran «datos insuficientes».

11. Plan de trabajo

Cronograma general del módulo (ya fijado por el curso):

- NF1 — Propuesta y anteproyecto (10%): 17/09–22/09/2026 — **cerrado**
- NF2 — Diseño, datos y planificación (25%): 23/09–14/10/2026 — **entrega el 14/10**
- NF3 — Desarrollo, integración y validación (35%, cuello de botella principal): 15/10–12/11/2026
- NF4 — Entrega, demostración y defensa (30%): 13/11–10/12/2026
- Cierre: 11/12–18/12/2026

Cronograma de NF3 y NF4 (propuesta de Marcos, revisada por Álvaro). El cronograma de NF2 ya se ha cumplido. Marcos no está del 28/10 al 09/11.

Fechas	Marcos	Álvaro
15/10–21/10	Esquema en PostgreSQL, Docker Compose, carga del corpus, `generar_turnos.py`; contrato de la API publicado hacia el 17/10	Guiones en JSON, módulo de métricas con tests y esqueleto del dashboard contra el contrato de la API
22/10–27/10	Backend completo: conversación (LLM simulado y Ollama), registro de turnos, puntuación 0/1/2, métricas, entrenamiento, predicción y tests del pipeline; `docker compose up` funcionando	Dashboard con métricas (historia 6)
28/10–09/11	Ausente	Dashboard sobre la API (alerta, evolución longitudinal y, si da tiempo, consola editable); frontend del chat sobre la API (a cargo de Álvaro)
10/11–12/11	Integración final y flujo completo	Integración final y flujo completo
13/11–26/11	Historias Could y memoria	Historias Could y memoria
27/11–10/12	Demo, ensayo y defensa	Demo, ensayo y defensa

Hitos: API cerrada y documentada hacia el 17/10; `docker compose up` funcional antes del 28/10; flujo completo el 12/11. **Dependencias:** el dashboard y el frontend dependen del contrato de la API. Si la semana del 22/10 se complica, lo primero que se mueve es Ollama a la vuelta de Marcos, porque con el modo simulado la API ya es usable.

Reparto de tareas: ver sección 1 (roles) y Anexo de reparto detallado ya acordado en NF1.

Riesgos técnicos: ver sección 13.

12. Estrategia de desarrollo

- **Flujo Git (branches) — convención propuesta:** `main` protegida (ya existe en el repo) + ramas `feature/<tarea-o-issue>` en kebab-case (p. ej. `feature/generador-corpus`, `feature/dashboard-alertas`, siguiendo el mismo patrón ya usado en `feature/guiones-y-`

`perfiles-simulados`), fusionadas por Pull Request. Sin rama `develop` intermedia — el equipo es de 2 personas, una capa extra de integración no aporta valor a este tamaño.

- **Cómo se integrará el código:** ninguna tarea se sube a `main` sin PR revisada por el otro miembro (regla ya acordada); toda tarea tiene una issue asignada; las decisiones técnicas relevantes se documentan en `docs/adr/`.
- **Cómo se validará funcionalidad:** revisión cruzada de PRs; ejecución manual del flujo completo (sesión → evento → métricas → predicción → dashboard) antes de fusionar cambios que toquen ese pipeline.
- **Testing básico — propuesta cerrada por Álvaro (las métricas son su área; el clasificador y su predicción por API, de Marcos):**
  - **Framework:** `pytest`, con `pytest-cov` para medir cobertura solo de los módulos de mayor riesgo (no se persigue cobertura total del proyecto).
  - **Tests unitarios de métricas:** (`features.py` solo construye las ventanas) una función por métrica (aMLU, LPP, IRC%, PEPE, %DET) con entradas y salidas conocidas. Caso de referencia (2 turnos de apertura + 3 de competencia): palabras 2, 3, 4, 3, 5 → aMLU = 3,4; latencias de E 5.200, 4.100, 6.000 ms → LPP = 5.200 ms; `iniciativa` 0, 0, 0, 1, 0 → IRC = 20 %; `score` 2, 1, 0 → PEPE = 1,0; x = 2, 1, 0, 2, 1, 0, 2, 2, 1 → %DET = 80,0 y la secuencia errática 2, 0, 1, 0, 2, 1, 1, 0, 2 → 33,3.
  - **Tests del pipeline de clasificación:** (a) el split train/test respeta la separación por `student_id` (ninguna sesión del mismo alumno simulado aparece en ambos conjuntos); (b) test de humo: un modelo ya entrenado, ante un vector de features bien formado, devuelve siempre una de las 3 clases válidas; (c) test de regresión de métricas: tras reentrenar, la media de la validación cruzada no cae por debajo de los umbrales de la sección 8 (F1-macro ≥ 0,75, precisión y recall de estancamiento ≥ 0,65, y al menos 0,05 de F1-macro por encima de la heurística); (d) comprobaciones antifuga automatizadas: el modelo con etiquetas permutadas queda en F1-macro ≈ 0,33 (≤ 0,40) y el modelo tramposo (competencia + sesión) al menos 0,10 por debajo del real; (e) la competencia y el nº de sesión no figuran entre las features, y los rangos de normalización y el tamaño de ventana se leen de `config.yaml` para que entrenamiento e inferencia no se desincronicen.
  - **Test ligero de dashboard:** dado un `session_data` fixture con etiqueta "Estancamiento" conocida, el bloque de alertas debe renderizar el mensaje de estancamiento (no el de progreso óptimo) — sin necesidad de un test end-to-end completo de UI.
  - **Dónde viven:** `tests/` en la raíz del repo, reflejando la estructura de `src/` (p. ej. `tests/test_metricas.py`, `tests/test_features.py`, `tests/test_classifier.py`).

13. Riesgos técnicos actualizados

Riesgo	Impacto	Probabilidad	Plan de mitigación
No hay datos reales de menores con TEA	Medio	Alta (es una certeza del alcance)	Corpus sintético/simulado, documentando la limitación explícitamente
Alcance demasiado grande para 2 personas en el calendario	Alto	Media	MVP descopado (ver alcance IN/OUT); Kafka/Spark documentados, no implementados
El modelo no alcanza métricas fiables con pocos datos sintéticos	Medio	Baja (ya medido: F1-macro 0,78 frente a 0,68 de la heurística)	Heurística simple como respaldo; umbrales cerrados sobre la media de la validación cruzada
Corpus demasiado fácil (acierto ~100 %, resultados no creíbles) o demasiado difícil (el modelo no supera a la heurística); umbral y ruido acoplados	Medio	Media	Barrido de ruido como análisis de sensibilidad, comprobaciones antifuga y rango esperado de F1-macro 0,75-0,82; ajustar `sigma_relativo` si sale ≥ 0,90 o < 0,75
El corpus sintético (cuello de botella de NF3) se retrasa desde NF2	Alto	Media	Recortar el alcance del modelo antes que el tiempo de NF4

Riesgo	Impacto	Probabilidad	Plan de mitigación
El modelo entrenado con datos sintéticos no es válido clínicamente	Bajo (fuera de alcance del curso)	Alta	Usarlo como prueba de arquitectura; reservar validación con centro real fuera del alcance de este curso
El menor desarrolla apego al chatbot o dependencia comunicativa unidireccional	Alto	Media	Límite de duración y de sesiones por semana, personaje sin lenguaje afectivo, uso decidido por el terapeuta y práctica entre sesiones de terapia (sección 9)
El menor rechaza al asistente	Medio	Media	Suspender el uso y derivar al terapeuta, que decide si adaptar el personaje o el guion (sección 9)
Ausencia de Marcos del 28/10 al 09/11, con el backend en su área	Medio	Alta (confirmada)	API cerrada, documentada y levantable con Docker antes del 28/10; modo de LLM simulado para trabajar sin GPU; Ollama se integra a su vuelta
El LLM genera respuestas inadecuadas o fuera de guion con un menor	Alto	Baja	Turnos basados en guiones cerrados y revisión de la salida del LLM antes de mostrarla
El prototipo no se puede reproducir en otro equipo (sin GPU o sin modelo descargado)	Medio	Baja	Docker Compose con versiones fijadas y modo de LLM simulado para ejecutar la demo en cualquier portátil

14. Repositorio del proyecto

Repositorio Git público, creado por Marcos Expósito Flox y accesible para ambos miembros:

https://github.com/MarcosExp/Social-Sim-Chat

Estructura exigida por la actividad:

```
repo/
 docs/
  NF2_Documentacion_y_Diseno_Tecnico.pdf
  arquitectura.png
  er.png
 src/
 data/
 environment/
 reports/
 README.md
 .gitignore
```

Estado del repositorio (13/10/2026). Las PR #2 y #1 están fusionadas en `main` (primero la #2, luego la #1). El repositorio contiene `config.yaml`, el generador del corpus (`src/generador/`), las features y la evaluación antifuga (`src/ml/`), `informe_metricas.json` en `reports/` con los valores por semilla y el cumplimiento de cada umbral, `arquitectura.png`, `er.png` (con `iniciativa`) y `cambios-generador-corpus.md` en `docs/`, el README con la sección «Arquitectura resumida» y `environment/` ya con su nombre definitivo (por ahora vacía: el `docker-compose` y el `.env.example` llegan en NF3). `docs/` alojará también el PDF de este documento.

README.md debe contener (obligatorio):

- Descripción del proyecto
- Integrantes
- Arquitectura resumida
- Cómo ejecutar (aunque aún no funcione)
- Tecnologías

Anexo A. Evaluación del clasificador y regla de alerta

Detalle de la medición de Marcos (PR #2, 10 semillas, 12.000 sesiones evaluadas). Es el soporte de los umbrales de la sección 8.

- **Heurística de referencia:** pendiente de PEPE en la ventana de 4 sesiones; por debajo de −0,02 por sesión es regresión, entre −0,02 y +0,02 estancamiento y por encima progreso (F1-macro 0,68, accuracy 0,75, recall de estancamiento 0,45).

- **Cumplimiento por semilla (10 semillas):** F1-macro ≥ 0,75: 7 de 10 (70 %); precisión de estancamiento ≥ 0,65: 8 de 10 (80 %); recall de estancamiento ≥ 0,65: 8 de 10 (80 %); ventaja sobre la heurística, etiquetas permutadas, alumnos detectados y retraso: 10 de 10; modelo tramposo y falsas alertas: 9 de 10. En particiones sueltas el cumplimiento baja al 68-76 %. Las tres semillas que fallan el F1 lo hacen por milésimas (0,744, 0,748 y 0,750). Es la confirmación de que los umbrales son válidos **sobre la media** y no sobre un corpus o una partición concretos.

- **Matriz de confusión** (10 semillas, 12.000 sesiones evaluadas; filas = real, columnas = predicho; orden estancamiento / progreso / regresión): [1.926, 533, 302], [498, 6.382, 151], [368, 245, 1.595]. Por clase: progreso, precisión 0,89 y recall 0,91; regresión, precisión 0,78 y recall 0,72.

- **Honestidad sobre cómo se fijaron:** F1-macro ≥ 0,75 y precisión ≥ 0,65 se propusieron en el borrador de NF2 **antes** de rehacer el generador. El recall se revisó de 0,80 a 0,65 **después de la primera medición**; la causa es el efecto de la transición (siguiente punto), medido y explicado, no un ajuste arbitrario. Con ≥ 0,70 el criterio incumpliría la mitad de las veces por la dispersión entre particiones.

- **Efecto de la transición:** en las 2-3 primeras sesiones tras el inicio del cambio el acierto es 0,52, porque la ventana de 4 sesiones aún ve el progreso anterior; a partir de la 3.ª sesión sube a 0,85. De ahí el recall de estancamiento moderado.

- **El modelo aprende la evolución, no el nivel:** las cinco pendientes suman el 62 % de la importancia (la mayor, la pendiente de %DET, 0,20); ninguna variable domina y la competencia no es feature.

- **Comprobaciones antifuga:** (1) *modelo tramposo* con solo competencia y nº de sesión: F1-macro 0,58 frente a 0,78 (exigido: al menos 0,10 por debajo); (2) *etiquetas permutadas*: F1-macro 0,32, es decir azar con tres clases (criterio ≈ 0,33; la accuracy no sirve aquí porque con `class_weight="balanced"` cae por debajo de la clase mayoritaria).

- **Análisis de sensibilidad (no es una validación):** al variar el ruido del generador (`sigma_relativo`) el F1-macro cae de 0,82 (0,01) a 0,78 (0,02, el elegido), 0,74 (0,03), 0,66 (0,05) y 0,60 (0,07). Con 0,03 ya incumple el umbral: **umbral y ruido están acoplados**, es decir, el criterio solo tiene sentido junto con el nivel de ruido declarado.

- **Criterio operacional de alerta (regla implementada):** el dashboard alerta cuando el clasificador predice **no-progreso (estancamiento o regresión) en 3 sesiones consecutivas**. Evita que una sesión ruidosa aislada dispare la alerta. Sustituye a la propuesta inicial, que solo contaba estancamiento y añadía una ventana de 5 sesiones con umbral del 60 %, no implementada. Se mide a nivel de alumno, con valores de referencia fijados con margen tras la primera medición (valores de diseño, no estándares clínicos):

Medida	Referencia	Medido
Alumnos con cambio que llegan a alertar	≥ 90 %	97 %
Retraso mediano desde el inicio del cambio	≤ 4 sesiones	3 (media 2,8; con la regla de 3 sesiones el mínimo habitual es 2)
Alumnos que progresan con alguna falsa alerta	≤ 15 %	9 %

Nota: en los alumnos que progresan con cambio de signo las métricas bajan pero la etiqueta de diseño sigue siendo "progreso", así que cualquier alerta ahí cuenta como falsa; el 9 % es, por tanto, algo pesimista.

Anexo B. Declaración de uso de inteligencia artificial y autoría de recursos

Uso de IA. Se ha utilizado Claude (Anthropic) como asistente en:

- La redacción y maquetación de esta documentación a partir de las decisiones del equipo, y en la revisión de coherencia entre secciones y con el repositorio.
- La verificación de las cifras del clasificador, replicando la validación con otra versión de scikit-learn.
- El código del prototipo de la pantalla de chat y del dashboard del repositorio paralelo de Soultech Labs, revisado y probado por Álvaro García Puche.
- Marcos Expósito Flox: apoyo en el generador de corpus, el pipeline de features y el entrenamiento del clasificador.

Las decisiones de diseño, el alcance y el contenido pedagógico son del equipo. La IA se ha usado solo como apoyo.

Autoría y recursos de terceros.

- Corpus: generación propia, sin datos de terceros (sección 8).
- Generador de corpus, pipeline de features y clasificador: Marcos Expósito Flox (github.com/MarcosExp/Social-Sim-Chat).
- Guiones, perfiles simulados, definición de métricas y dashboard: Álvaro García Puche.
- Bibliotecas de código abierto: scikit-learn, numpy, pandas, PyYAML, FastAPI, Streamlit, SQLAlchemy y PostgreSQL.
- Modelo de lenguaje local: Qwen3 14B (Alibaba Cloud, licencia Apache 2.0), servido con Ollama.